

Odd Sum Subarrays

Company: Deloitte

Round: Online Assessment (OA)

Year: 2024

Difficulty: Medium

Topic: Arrays, Prefix Sum, Parity

Source: [LeetCode 1524 - Number of Sub-arrays With Odd Sum](#)

Problem Description

You are given an array of integers. A subarray is defined as any contiguous segment of the array. The objective is to determine the total number of subarrays whose sum of elements is odd.

Input Format

- The first line contains a single integer N ($1 \leq N \leq 10^5$), denoting the size of the array.
- The second line contains N space-separated integers, each in the range $[-1000, 1000]$.

Output Format

- Print a single integer representing the total number of subarrays whose sum is odd.

Sample Input / Output

Example 1

Input:

5

1 2 3 4 5

Output:

9

Example 2

Input:

3

1 1 1

Output:

4

Explanation

In Example 1, among all 15 possible subarrays of $[1, 2, 3, 4, 5]$, exactly 9 yield an odd sum — including $[1]$, $[1, 2]$, $[3]$, $[2, 3]$, $[4, 5]$, $[1, 2, 3, 4]$, $[3, 4, 5]$, $[2, 3, 4, 5]$, and $[1, 2, 3, 4, 5]$.

In Example 2, among all 6 possible subarrays of [1, 1, 1], exactly 4 yield an odd sum — each of the three individual elements, and the full array, whose total is 3.

Approach to Solve This Problem:

A brute-force strategy that evaluates the sum of every subarray individually requires $O(N^2)$ time, which is computationally infeasible for N as large as 10^5 . The problem, however, can be reduced to an observation about parity: only whether a prefix sum is odd or even matters, not its exact numerical value.

If $\text{prefix}[i]$ denotes the sum of the first i elements of the array, then the subarray spanning indices $i + 1$ through j has an odd sum precisely when $\text{prefix}[i]$ and $\text{prefix}[j]$ differ in parity — one even and one odd. This observation allows the problem to be solved in a single linear pass by maintaining running counts of even and odd prefix sums rather than recomputing subarray sums directly.

1. **Maintain the running parity:** Traverse the array while tracking the parity (odd or even) of the cumulative sum, toggling it whenever the current element is odd.
2. **Maintain two running counters:** Keep separate counts of how many prefix sums encountered so far — including the empty prefix, whose sum is 0 — are even and how many are odd.
3. **Count subarrays ending at the current index:** If the running sum is currently odd, every even prefix encountered earlier contributes one valid odd-sum subarray; if the running sum is currently even, every odd prefix encountered earlier contributes one.
4. **Update the counters:** After processing each element, increment the counter corresponding to the current prefix parity so that subsequent indices can reference it.

Java Solution:

```
import java.util.Scanner;

class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int[] arr = new int[n];
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        long countEven = 1; // accounts for the empty prefix (sum = 0)
        long countOdd = 0;
        boolean isOdd = false;
        long answer = 0;

        for (int i = 0; i < n; i++) {
            if (arr[i] % 2 != 0) {
                isOdd = !isOdd;
            }

            if (isOdd) {
                answer += countEven;
                countOdd++;
            } else {
                answer += countOdd;
                countEven++;
            }
        }

        System.out.println(answer);
    }
}
```

Solution:

- 1. Read the input:** `main()` reads `N` via Scanner, followed by `N` integers into `arr`, in accordance with the Input Format.
- 2. Initialize the counters:** `countEven` is initialized to 1 to represent the empty prefix (`sum = 0`, which is even), `countOdd` is initialized to 0, `isOdd` tracks the parity of the running prefix sum, and `answer` accumulates the final count.
- 3. Update the running parity:** For each element, if `arr[i]` is odd, `isOdd` is toggled, reflecting the fact that adding an odd value inverts the sum's parity while an even value leaves it unaffected.
- 4. Determine subarrays ending at the current index:** When the running sum is odd, each previously recorded even prefix combines with it to form a valid odd-sum subarray, so `countEven` is added to `answer`. When the running sum is even, each previously recorded odd prefix contributes instead, so `countOdd` is added.
- 5. Update the counters and produce the output:** The counter corresponding to the current parity is incremented, and once the traversal is complete, `answer` is printed, satisfying the Output Format.

Complexity Analysis:

Time Complexity: $O(N)$

The array is traversed exactly once, with each element processed in constant time through a parity check and counter update. This results in an overall linear time complexity.

Space Complexity: $O(1)$

Excluding the storage required for the input array itself, the algorithm relies on a fixed set of scalar variables — `countEven`, `countOdd`, `isOdd`, and `answer` — resulting in constant additional space usage.